Computing (2026) 108:110
https://doi.org/10.1007/s00607-026-01701-5

REGULAR PAPER

Consistency challenges in event-driven microservices: a literature review on data, process, and system evolution

Joaquim Luz Bação[1] · **Sérgio Guerreiro**[1,2]

Received: 29 January 2026 / Accepted: 16 June 2026 / Published online: 29 June 2026
© The Author(s) 2026

Abstract

As event-driven microservice architectures become more prevalent in modern software systems, especially those integrated with business processes, the issue of consistency across distributed components has emerged as a central challenge. Despite extensive adoption, there remains no clear consensus on what microservices are or how consistency should be defined and maintained across such systems. This paper addresses the fragmented understanding of consistency in microservice architectures, aiming to identify how it is conceptualized in the literature and what strategies are used to manage it effectively. The methodology follows the systematic review procedures originally proposed by Kitchenham [1]. The review revealed three dominant categories of consistency: data consistency, process consistency, and system evolution consistency. Each type involves distinct approaches that address specific challenges documented in the literature. These concerns are often addressed in isolation, resulting in a fragmented research landscape. The selection of digital libraries and the exclusion of gray literature may have limited the scope of the review. Future work could expand the dataset and further validate the proposed model. Other categories that are deemed relevant may also be added. Three main originality aspects are identified in this paper as follows: i) the categorization of consistency into the three types; ii) the organization of the main approaches used to address each type of consistency; and iii) the discussion of the relationship between consistency types and the selection of appropriate approaches to guarantee each type.

Keywords Microservice · Microservice architecture · Event-driven · Business process · Consistency

Joaquim Luz Bação and Sérgio Guerreiro contributed equally to this work.

Extended author information available on the last page of the article

1 Introduction

As enterprises increasingly adopt event-driven microservice architectures integrated with complex business processes, ensuring consistency across their distributed components emerges as a critical concern [2]. Microservice architectures are software design approaches where applications are composed of small, loosely coupled building blocks, each focused on a single business capability, that communicate with each other over lightweight protocols [3]. These services are independently developed, deployed, and released, enabling high scalability, resilience, and flexibility [4, 5]. This shift is often driven by the need to overcome the rigidity and maintenance challenges of monolithic systems, motivating organizations to embrace architectures that better support rapid change and system evolution [6]. For these reasons, understanding and managing consistency becomes crucial to ensure that the expected benefits of microservices are realized in practice.

However, the rapid evolution of the field, combined with the lack of a predominant architectural modeling language [7], makes it difficult to obtain a comprehensive understanding of how microservices are designed and managed. Architectural representations often vary depending on the specific challenges being addressed, such as deployment, communication, or coordination, which further complicates consistent modeling and analysis across systems [8]. The lack of a shared conceptual framework is particularly evident in how microservice consistency is handled. Even though it is a cornerstone of distributed architecture, the literature remains siloed, with studies approaching the problem from disparate, uncoordinated angles. Discussions of consistency vary in scope and terminology, with different works emphasizing different types, including data, processes, and system evolution consistency [9, 10].

Therefore, the purpose of this literature review is to investigate how consistency is understood, approached, and managed in event-driven microservice architectures, particularly when these systems are supporting the business processes operation. This paper aims uncovering which types of consistency are most frequently addressed in the literature and examining the strategies and tools used to address them, as well as limitations and fragmentation in those practices. Rather than assuming predefined categories, the review pays attention to how consistency issues are framed across different studies, with the goal of defining a clearer, more structured understanding of the problem space. Compiling and synthesizing existing research in this way is essential to identify gaps and guide future work in such a fragmented and rapidly evolving field [1].

This paper is organized as follows. Section 2 introduces the key theoretical concepts necessary for understanding the review. Section 3 positions the paper against related work. Section 4 describes the methodology employed for the literature review. Section 5 presents the characteristics of the reviewed studies, including the distribution of key concepts and the rationale for their thematic grouping. Section 6 details the findings of the review, addressing the research questions outlined earlier. Section 7 discusses these findings in the context of existing literature. Finally, Section 8 provides the paper's conclusions and implications.

2 Background

This section presents the theoretical background necessary to support the review. It begins by outlining the motivations behind the transition from monolithic to microservice architectures, then introduces the core principles and commonly accepted characteristics of microservices. Finally, it introduces the general principles of event-driven architecture and business process modeling, which form the broader context in which microservices are increasingly being applied.

2.1 Monolithic-based systems transition to microservices

Monolithic architectures have traditionally been used to develop software systems as a single, unified application. In these systems, all components are interdependent, making even minor changes potentially risky and time-consuming.

As systems grow and evolve, monolithic structures often become difficult to maintain. Frequent updates, feature additions, and bug fixes can deteriorate the architecture, leading to tightly coupled code that is fragile and hard to scale [6, 11].

Therefore, microservice architectures emerged as a response to these limitations, breaking down applications into independent, fine-grained services, each responsible for a distinct function. This separation of concerns improves maintainability and supports parallel development across teams.

One of the key advantages of microservices is scalability. Individual services can be scaled based on their specific performance requirements, rather than scaling the entire application [11]. This targeted scaling reduces infrastructure costs and allows organizations to respond more efficiently to changing workloads [6]. Beyond technical benefits, microservices support business agility. By enabling faster adoption of new technologies and easier integration with third-party APIs, organizations can better respond to changing customer needs [12].

2.2 Business processes

A business process represents a central tool for structuring and interpreting the interconnections between activities that lead to a market deliverable [13]. Furthermore, the broader field of business process management focuses on improving corporate performance by supporting the whole lifecycle of a company's processes, ranging from identification and modeling to actual operations [14]. In complex scenarios, these processes act as choreographies that define exactly how work is done, taking into account all involved participants and systems [15].

The modeling of business processes is crucial, and languages such as the Business Process Model and Notation (BPMN) are widely used for this purpose [11]. BPMN offers a visual notation that can be interpreted directly by both analysts and developers, allowing business stakeholders to specify business process workflows that developers subsequently implement [3]. This makes BPMN an important bridge between conceptual process design and executable system behavior.

2.3 Workflows

A workflow is primarily defined as the automation of a business process according to a set of procedural rules [16]. These models are often represented as directed graphs (e.g., BPMN diagrams) that can be decomposed into smaller sub-graphs and "recomposed" to create new, verified processes without introducing modeling errors [17]. In modern distributed architectures, such as microservices, workflows provide an abstraction that coordinates and sequences activities across multiple independently deployed services [18].

They enable decentralized services to be integrated into cohesive, end-to-end business processes, offering explicit control over execution order, state management, error handling, and long-running interactions [19]. In complex environments where multiple workflows may run simultaneously, this explicit control must also be conflict-aware to prevent concurrent processes from competing for the same resources [20]. By externalizing coordination logic from individual services, workflows enhance maintainability and observability while preserving microservice autonomy. As a result, workflows act as a practical mechanism for implementing the business processes modeled at the conceptual level.

2.4 Event-driven architecture

Event-driven architecture is a software design pattern in which components communicate by producing and responding to events [21]. Instead of relying on synchronous calls, systems built on this architecture use an event broker, or event bus, to decouple components. Each service can publish events and subscribe to those it is interested in, enabling asynchronous and reactive interactions [10, 22]. However, the presence of a centralized BPMN orchestration engine does not by itself make a microservice system event-driven. Systems coordinated through a process engine may still rely predominantly on synchronous process control, whereas event-driven behavior emerges only when runtime coordination between services occurs mainly through asynchronous and decoupled event exchange [8, 23].

In microservice-based systems, this pattern is particularly relevant, as it supports loose coupling, high scalability, and independent deployment [24]. Figure 1 illustrates how microservices interact through a publish-subscribe mechanism coordinated via an event broker.

This design provides strong support for scalability and flexibility, making it well suited for distributed systems. Zuki, et al. [25] describe implementations where event brokers significantly improve system throughput and response time by managing message distribution across services. However, Woodside [26] notes that while pure EDA models are naturally scalable, real-world "hybrid" systems that retain synchronous legacy components require specialized performance modeling to prevent bottlenecks.

Together, event-driven architecture and business process modeling offer foundational principles for building responsive and well-aligned microservice systems. While event-driven architecture provides the underlying communication mechanism

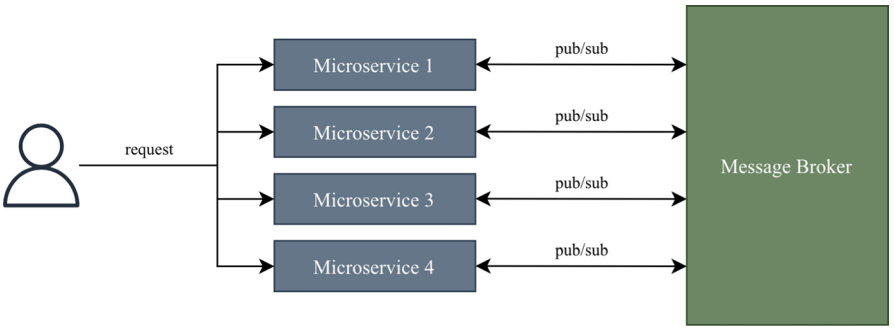


Fig. 1 Publish-subscribe (pub/sub) communication between microservices via a central Message Broker in an event-driven architecture. Adapted from [25] by the authors

for loosely coupled interactions, business process models ensure that system behavior remains aligned with meaningful organizational objectives.

2.5 ACID transactions

The ACID model, Atomicity, Consistency, Isolation, and Durability, captures the core properties that relational databases use to ensure correctness and reliability in transactional systems [27]. These guarantees were originally designed for centralized, monolithic architectures, where enforcing strict transactional behavior is comparatively straightforward [28].

1. *Atomicity:* Ensures that a transaction executes as a complete unit, where all operations succeed together or are entirely rolled back if any part fails. This prevents partial updates and maintains a clean, predictable system state [27].
2. *Consistency:* Requires that each transaction transitions the system from one valid state to another, preserving all defined rules and constraints. A transaction may change the system's state unless it is read-only, but it must never violate established invariants [27].
3. *Isolation:* Ensures that concurrently executed transactions do not interfere with each other. Each transaction should behave as if it were running alone, unaffected by operations occurring in parallel [28]. This property is often associated with serializable execution [27]. While local isolation can be enforced within a single database, ensuring the same behavior across distributed components is significantly more challenging [29].
4. *Durability:* Guarantees that once a transaction is committed, its effects are permanently recorded. Even in the event of a system failure, the resulting data must remain intact and recoverable [28].

Although these ACID properties were designed around traditional relational databases, they continue to serve as a key reference point when evaluating data reliability in modern architectures. Even emerging approaches, such as Service Proxy Transaction Management, which aims to provide ACID guarantees at snapshot isolation, still

build on the principles defined by the ACID model. Ultimately, the goal of ACID remains the same: to preserve data accuracy, integrity, and stability in transactional systems.

2.6 Saga pattern

The Saga pattern is a mechanism for handling distributed transactions in microservices architectures, where a transaction is decomposed into a sequence of sub-transactions that may interleave with others [9]. In microservices, a Saga represents a global transaction composed of multiple local transactions ($T_1, T_2, \ldots, T_n$), each executed within a single service [28].

Because traditional rollback is not feasible across distributed systems and atomic commit protocols like 2PC reduce responsiveness in long-lived transactions [30], Sagas rely on compensating actions ($C_1, C_2, \ldots$) that semantically undo previously completed steps when a failure occurs [31]. These compensations are applied in reverse order, ensuring the system either completes all steps or returns to a consistent state [28, 29].

Two main variants exist: choreographed Sagas, where services coordinate through events without a central controller [29], and orchestrated Sagas, where a coordinator directs the workflow [9]. Unlike ACID transactions, Sagas do not provide isolation and instead operate under eventual consistency, maintaining ACD properties while relaxing strict guarantees to suit distributed environments [31, 32]. This makes the Saga pattern an effective approach for restoring consistency in distributed systems where global ACID transactions are impractical.

3 Related work

The growing body of systematic reviews and empirical studies on microservice architectures have mapped the field from several angles, yet none addresses consistency from the combined perspective of data, process, and system evolution in the context of event-driven systems integrated with business processes.

Söylemez et al. [33] reviewed 85 primary studies and identified nine broad challenge categories in microservice architectures, among them data management and consistency, service discovery, and orchestration. Despite its breadth, the review subsumes consistency within data management without decomposing it into distinct types, does not examine event-driven architectures as a specific coordination paradigm, and excludes business process integration entirely.

Laigner et al. [34] triangulate a systematic literature review with open-source application analysis and a survey of over 120 practitioners to characterize data management challenges in microservices, covering eventual consistency, cross-service querying, and schema evolution. Their scope is deliberately bounded to the data dimension, leaving process consistency, system evolution consistency, and business process integration outside the inquiry.

Laigner et al. [35] provide the first empirical characterization of event management challenges in microservices, mining over 8,000 Stack Overflow questions. The

practitioner difficulties they uncover, including safe event publication, out-of-order delivery, and schema evolution, map naturally onto the three consistency types examined here. However, the study derives its findings from developer discourse rather than academic synthesis, organizes them around event management properties rather than consistency as a structuring concept, and does not engage with business process models.

Alshuqayran et al. [36] conducted one of the earliest systematic mapping studies of microservice architecture, surveying the literature to characterize adoption patterns, quality attributes, and architectural decisions. The study provides a useful baseline for understanding how the field has been mapped in general terms, but consistency is not treated as a structured analytical concern and business process integration does not feature in the analysis.

Each of these works engages with some dimension of the problem addressed here, yet none proposes an explicit organization of consistency into distinct types or examines how business process concepts mediate the solutions proposed for each. The present review fills that gap.

4 Research methodology

This literature review was conducted to gather recent research on event-driven microservice architectures that incorporate business processes, with a particular focus on how such systems ensure consistency.

To the best of the authors' knowledge, no prior literature reviews have addressed the intersection of these three aspects comprehensively. Existing reviews tend to focus on individual components, but not on their combined implementation and implications for consistency guarantees.

4.1 Methodology

The approach taken in this paper draws on the systematic review guidelines originally proposed by [1], which have become well-established within software engineering research. These guidelines outline a structured process consisting of three core phases.

1. *Planning,* which involves defining the purpose of the review, designing the review plan, and formulating the research questions that guide the investigation,
2. *Conducting,* where the review plan is put into action through the identification, selection, and detailed examination of relevant literature, followed by data extraction and analysis, and
3. *Reporting,* focused on compiling the key findings, addressing the research questions, and communicating the insights gained.

This review's methodology follows these steps closely to ensure a comprehensive and transparent examination of the literature. The detailed actions taken at each stage are summarized in Table 1.

Table 1 Literature review steps and sub-steps

#	Steps	Sub-steps
1	Planning	State objectives and baseline research questions
		Design research protocol
		• Define inclusion criteria
		• Define databases
		• Define query strings
		• Describe selection process
2	Conducting	Perform paper selection
		Describe and analyze results
3	Reporting	Systematize findings
		• Synthesize, classify and compare studies
		• Perform exploratory discussion
		• Propose framework for future research
		Present findings and answer research questions

Adapted from [1] by authors

According to [1], literature reviews aim to consolidate existing knowledge, uncover gaps or unresolved issues in the research landscape, and provide a foundation for further investigations. Therefore, the aim of this paper is to systematically gather evidences on consistency solutions, in event-driven microservice architectures, highlighting areas where challenges remain, and suggesting directions for future work.

4.2 Research questions

This review aims to investigate key challenges in event-driven microservice architectures, particularly focusing on how consistency is maintained and the role of business processes within these systems. To provide a clear and focused examination it is structured around the three following research questions.

RQ1: What are the current architectural approaches to ensure consistency in event-driven microservices?

RQ2: How do these approaches incorporate business process concepts?

RQ3: What limitations and challenges are reported regarding these approaches?

4.3 Protocol design

An automated search was conducted on the Web of Science digital library, focusing on titles and abstracts. The Web of Science Core Collection is a curated index of high-impact journals, books, and conference proceedings, selected according to established quality and citation criteria. It serves as a comprehensive, multidisciplinary database that supports systematic and reliable literature retrieval. No publication date limitation was imposed. Hence, it is implicit that the date limit for the publication of documents corresponds to the date the query was run on (November 21, 2025). Finally, the language was limited to English, as the majority of research in this field is published in English.

The search strategy was iteratively refined by adjusting keywords to retrieve a manageable and relevant set of studies. The search string used to query the database is presented as follows.

TI=("microservice") AND AB=((("architectur*" OR "orchestrat*" OR "choreograph*") AND (("event driven" OR "event based" OR "event stream*") OR "consistency")) OR ("business process*" OR "BPM*" OR "process model"))*

The search string was constructed to always include terms related to microservices and grouped into three main segments. The first focused on architecture together with event-related terms, the second combined architecture with consistency-related terms, both corresponding to RQ1 and RQ2. The third segment targeted business process-related terms, which are more closely tied to RQ3. This search returned a total of 119 documents.

4.3.1 Describing the selection process

The selection process was divided into four stages.

1. *Searching (using the specified string query),*
2. *Screening abstracts,*
3. *Screening conclusions, and*
4. *Screening of full-text documents.*

The number of valid documents was tracked throughout each stage of the selection process. The initial search returned 119 documents. In stage 2, abstracts were reviewed, and 62 documents were immediately discarded due to clear irrelevance. Of the remaining 57, 23 lacked sufficient information in the abstract to determine their relevance and were moved to stage 3, where their conclusions were reviewed. At this stage, 17 documents were excluded, leaving a total of 40 documents. Finally, these were fully read in stage 4, during which 3 were discarded. This process resulted in 37 documents being selected for full analysis, summarization, and inclusion in the final review. Additionally, one further document was identified through snowballing, bringing the total to 38. This selection process can be visualized in Figure 2.

4.3.2 Criteria for document selection

The screening of abstracts, conclusions and full texts of the reviewed documents was conducted according to predefined selection criteria, outlined in detail in Table 2. These criteria ensured that only studies directly relevant to the research focus on microservice architectures, business process integration, and their challenges were included in the review.

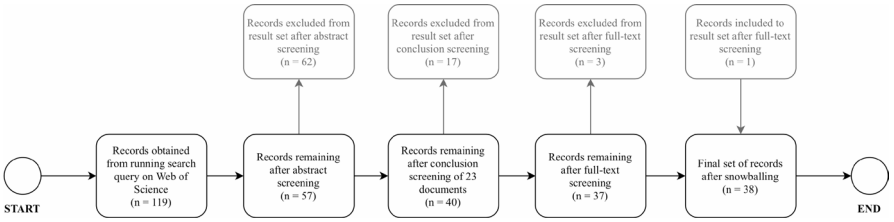


Fig. 2 Document selection process

Table 2 Selection criteria applied during screening.

Assessment	Criteria
Yes	Proposes or evaluates microservice architectures
	Integrates business process concepts into microservice design
	Addresses challenges related to microservice architectures
No	Discusses microservices generically without architectural focus
	Concentrates solely on decomposing monolithic systems into microservices without considering architectural consistency or business process integration
	Focuses on tools or techniques unrelated to the research questions, including testing or deployment

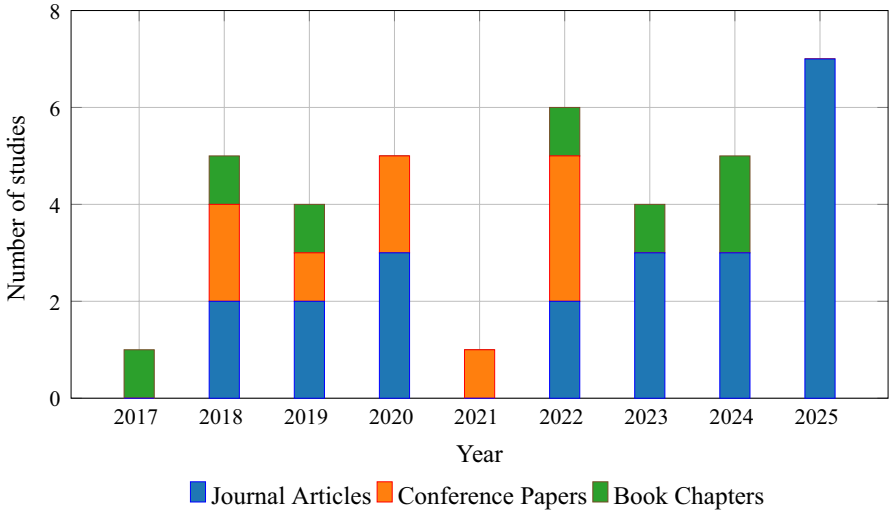


Fig. 3 Distribution of selected studies across publication type and year

5 Results

5.1 Profile of selected studies

The distribution of studies across the years and according to their source type (journals, proceedings, or book chapters) is illustrated in Figure 3. None of the studies included in this analysis were published before 2017, highlighting the relatively recent emergence and adoption of microservices, and the majority of publications reviewed came from 2025, further supporting this trend. The limited presence of 2021 studies does not indicate a lack of publications that year; rather, while some appeared in the initial screening, only one met the inclusion criteria.

5.2 Common concepts and contributions

While reading the full texts, we developed an iterative matrix of concepts and contributions. Some elements, such as business processes and novel architectures, were defined *a priori* based on the research questions, while additional contributions were added during full-text review as they emerged as relevant. When new contributions were identified, previously reviewed studies were re-examined to assess whether they also addressed these contributions. Contributions that appeared only rarely across the studies were sometimes discarded to maintain focus on the most relevant aspects.

As the matrix took shape, three semantically coherent clusters emerged from observing which concepts co-occurred repeatedly across studies and which problems they collectively addressed. Rather than being imposed in advance, these groupings reflect three qualitatively distinct questions that the literature addresses when reasoning about consistency in microservice systems: whether the data held across distributed services is correct, whether the business process executes as intended across those services, and whether the system remains coherent as its services and interactions evolve over time.

The first cluster groups *data consistency*, *ACID properties*, and *Saga pattern*, addressing consistency management and transactional reliability. The second cluster groups *business processes*, *event-driven approaches*, *orchestration*, *choreography*, *workflows*, and *graphs*, relating to event sequencing, coordination, and behavioral correctness across microservice systems. The third cluster groups *service discovery*, *service registries*, *graphs*, and *evolution*, reflecting mechanisms for locating services or adapting the architecture in response to change. Two additional concepts, *novel architectures* and *frameworks and other tools*, were tracked separately as they cut across all three clusters and relate directly to RQ1.

The frequencies of these concepts and contributions across the reviewed studies are summarized in Table 3. To distinguish the coverage each concept in the table is marked as either informative, indicating that it discusses the concept in a substantive and analytical manner, or briefly mentioned, meaning it is acknowledged but not explored in detail. The Total row aggregates these occurrences, showing how many studies engage with each concept to any extent. Together, these classifications and counts offer a quantitative perspective on the themes most emphasized in the field, forming the basis for the subsequent analysis of the research questions.

5.3 Analysis of reviewed studies

The reviewed studies were categorized into five thematic groups based on their primary contributions as identified through the analytical matrix. Rather than representing formal findings, this classification serves as an auxiliary organizational step, a way to group studies by their main focus so that patterns could be more easily identified and the research questions more systematically addressed in the subsequent analysis. These five groups are distinct from the three semantic concept clusters described above, which operate at the level of individual concepts rather than whole studies and form the basis for the consistency types examined in the findings.

1. *Data consistency:* The studies in this group examine the challenges and strategies for maintaining data consistency in microservice architectures. They discuss the role of ACID properties, event-driven communication, transactional patterns such as Saga, and middleware-based solutions as means to achieve eventual consistency and enhance system resilience.
2. *Service discovery:* This group addresses mechanisms for dynamic service location, communication, and workflow management. The studies explore flexible orchestration and choreography approaches that enhance adaptability and fault tolerance in evolving microservice environments, as well as graph-based methods for service location. Service evolution is a recurring theme across the group, while service registry appears in some studies as a complementary element, though not a defining characteristic.
3. *Novel architectures for consistency:* Contributions in this group surge from, as the name indicates, studies that propose innovative architectural solutions to support consistency, coordination, and decentralized evolution. Approaches include hierarchical brokers, containerized event-driven designs, reactive architectures, and hybrid orchestration–choreography models, often aimed at improving scalability, error handling, and process comprehensibility.
4. *Frameworks and other tools:* This group covers other approaches to microservices that were not necessarily new architectures, so development support, like modeling methods, automated code generation, runtime platforms, and visualization tools. These contributions facilitate microservice adoption, bridge business and technical concerns, and improve system comprehension, without redefining the architectural paradigm.
5. *General challenges and empirical studies:* This group includes studies that did not focus in any of the concepts from the matrix, but were still relevant as they highlight broad challenges in the design, implementation, and evolution of microservices, including alignment with business processes. It provides context for why microservices are adopted and the practical difficulties faced in distributed systems.

The groups were not established through rigid classification criteria. Instead, each document was assigned according to its primary contribution within the context of this review, even when multiple themes were addressed simultaneously. Consequently, some studies may appear in groups whose labels do not fully encompass the entirety of their contributions, but that more accurately reflect the document's central contribution as interpreted by the authors. For instance, Fan et al. [27] do not explicitly employ the term data consistency; however, consistency constitutes a central concern throughout the work, which motivated its inclusion in the data consistency group. In contrast, Monteiro et al. [18], who proposed Beethoven, a novel event-driven orchestration platform, were classified under the service discovery group rather than novel architectures, as the service discovery dimension was considered the platform's most distinctive contribution within the scope of this review. The studies in each group are presented sequentially in Table 3.

Table 3 Concepts and contributions of the reviewed studies

Studies	DC	ACID	SAGA	NA	FW	BP	ED	ORC	CHR	WF	SD	SR	GR	EV
[9]	★	☆	★			★	★	★	★	★	☆	★		
[27]	★	★								★				
[28]	★	☆	★		★			★						
[31]	★	☆	★											
[37]	★													
[38]	★	☆						★	☆					
[23]	★		★				★	★	★					
[16]					★	★	★	★	☆	★	★		★	★
[18]				★		★	★	★		★	★	★		★
[22]						★	★	★			★			☆
[39]					★					★	★	★		★
[3]				★		★	★	★	★	★	★	★	☆	
[13]				★		★			★		★			★
[25]	★			★			★			☆			★	
[40]	★			★			★							
[41]				★			★	☆	★	★	☆		☆	
[42]				★										
[24]	★			★			★							
[43]	★	☆	★	★			★	★	☆				★	
[44]	★			★			★	★			☆			
[45]				★					★					★
[10]			☆		★								★	
[11]					★	★	★			★	★	★		
[19]					★	★		★		★			★	
[29]		★	★		★			☆	☆					
[46]	☆				★			☆			☆			
[47]					★	★								

Table 3 (continued)

Studies	DC	ACID	SAGA	NA	FW	BP	ED	ORC	CHR	WF	SD	SR	GR	EV
[48]					★	★			★					
[49]					★					☆			★	☆
[50]					★		★							☆
[4]	☆													
[5]					★		★		★					
[7]								☆			★		★	
[8]	☆					★	★							
[12]						★					★			
[51]						★								
[52]								☆	☆					☆
[53]	★						★							
Total	15	7	7	11	12	13	16	16	13	11	13	5	9	8

★ Informative, ☆ Briefly mentioned. *DC* Data consistency, *ACID* ACID properties, *SAGA* Saga pattern, *NA* Novel Architecture, *FW* Frameworks and other tools, *BP* Business process, *ED* Event-driven, *ORC* Orchestration, *CHR* Choreography, *WF* Workflows, *SD* Service discovery, *SR* Service registry, *GR* Graphs, *EV* Evolution

6 Findings

This section presents the findings derived from aggregating the reviewed studies into thematic groups according to the ways they contribute to the research questions. During synthesis, distinct categories of consistency emerged from the literature, some explicitly identified by authors, others implicit in proposed solutions, and, therefore, labeled here for analytical clarity. Because many studies address consistency-related concerns without using a common vocabulary, it is important to first clarify the separation of these consistency types and the scope of each before answering the research questions.

The findings that follow therefore offer a structured overview of i) the different kinds of consistency evident in the literature; ii) how these are addressed in event-driven microservice designs; iii) the extent and manner in which business-process concepts are integrated into those solutions; and iv) the principal limitations that hinder effective alignment between architectural mechanisms and business needs.

6.1 From concept clusters to consistency types

The three semantic clusters identified in the concept matrix served as the analytical basis for the consistency types emerging in this review. Each cluster reflects a distinct dimension of consistency concern: the first addresses challenges of state correctness across distributed data stores, the second addresses behavioral correctness in the coordination and sequencing of activities across services, and the third addresses the preservation of coherent system behavior as services and their interactions evolve. These three dimensions defined the three consistency types examined in the findings: *data consistency*, *process consistency*, and *system evolution consistency*. Graphs appear in both the second and third clusters, as graph-based representations serve both to reason about process flows and to track structural dependencies during system evolution.

Studies were assigned to the type whose cluster they most substantively engaged with, as recorded in the matrix. Studies from the general challenges group were not assigned to any consistency type, as their primary contribution was contextual rather than the proposal or evaluation of a concrete consistency mechanism, which is the central focus of this review's research questions.

For most studies, the dominant cluster was evident from the matrix. A small number of borderline cases, where engagement was distributed across two clusters or concentrated in concepts shared between them, were solved through joint authors discussion examining each paper's central research contribution. For instance, [16] engaged substantively with both process-related concepts such as orchestration and workflows and evolution-related concepts such as service discovery and evolution, but its central contribution, the declarative composition of goal-driven workflows in dynamically changing service environments, was judged to be primarily concerned with system evolution consistency. Similarly, [25] engaged comparably with data and process concepts, but its core focus on event-log synchronization and fault tolerance between containerized services placed it in the data consistency type. Table 4 summarizes the resulting distribution.

Table 4 Distribution of reviewed documents by consistency type

Type	Documents	# Docs
Data consistency	[9, 24, 25, 27–29, 31, 37, 38, 40, 43, 44]	12
Process consistency	[3, 8, 10, 19, 22, 41, 50]	7
System evolution consistency	[13, 16, 18, 39, 45, 49]	6

6.2 Characterization of consistency in microservice architectures

6.2.1 Data consistency

Data consistency emerged as the most frequently emphasized challenge in the reviewed literature. Maintaining a coherent view of state across loosely coupled, event-driven services proves particularly fragile when systems experience failures, when client requests span multiple services (the "chain of calls"), and during full-application backup operations [37, 38]. These scenarios expose both transient and persistent divergence of state: partial executions, missed or duplicated events, and inconsistent snapshots can all lead to incorrect business outcomes if not detected and reconciled.

Patterns such as Saga are frequently discussed in this context because they make transactional boundaries explicit and surface compensation needs when distributed steps fail [28]. Beyond these operational considerations, Fan et al. [27] highlight that, rather than enforcing strict ACID guarantees, event-driven architectures rely on reliable event propagation and eventual state convergence to maintain coherence across distributed services. Thus, data consistency in event-driven microservices encompasses not only the preservation of correct state after individual operations, but also the coordination of end-to-end behavior across distributed transactions, backups, and recovery processes in systems designed for loose coupling and high scalability.

6.2.2 Process consistency

Process consistency centers on ensuring that distributed business processes and workflows execute correctly and predictably across multiple independent microservices. Although many studies discuss this phenomenon, they often do so under the broader notion of consistency without giving it a specific name. We adopt the term **"Process Consistency"** to explicitly refer to the workflow-oriented aspects of consistency, highlighting the need to preserve coherent business behavior across distributed, event-driven microservices. Orchestration and choreography arise as the main coordination models for this concern.

- Orchestration provides centralized control over service interactions and is recommended when a higher level of management is needed to handle complex data flows [41]. It simplifies specifying and processing coordinated sequences but can reduce flexibility for processing intermediate states. Moreover, the orchestrator (often called the coordinator) must be highly available and not become a single point of failure, resulting in increased system complexity and affected reliability and availability [38].

- Choreography favors decentralized collaboration where multiple parties interact without any single actor taking responsibility for centralized control [41]. BPMN choreography diagrams are useful for visualizing these interactions, emphasizing message exchanges and the collaboration among participants rather than a central controller [48]. When business activities are executed in a distributed fashion, as between departments or independent partners, choreography often becomes more convenient because it foregrounds inter-partner message exchange and decentralized coordination [13].

The practical difficulty of maintaining process consistency grows in event-driven microservice landscapes where flows are often implicit and hard to trace [50]. BPMN and other business-level specifications are not always well integrated with run-time architectures, leaving a gap between business logic and execution semantics [8]. As microservice ecosystems evolve, implicit dependencies, decentralized coordination, and stateful interactions introduce uncertainty that can propagate inconsistencies if not addressed [10, 41]. Thus, process consistency must be considered at the end-to-end process level, not only ensuring individual service correctness but also guaranteeing predictable, coherent progression of business processes across distributed services.

6.2.3 System evolution consistency

System evolution consistency addresses the challenge of maintaining coherent behavior and functional integrity as microservices and their processes continuously evolve. This dimension becomes critical when local changes propagate across services, as uncoordinated modifications can introduce inconsistencies, break workflows, or compromise data states [13]. The complexity of evolving systems, including dynamic service composition and frequent updates, increases the risk of diverging states and unpredictable behavior, making maintainability and robust coordination essential for preserving system-level consistency.

Event-driven architectures, where services and workflows interact asynchronously, further complicate this form of consistency. Approaches such as modeling microservices and their interactions as directed graphs or as fragmented global process models highlight the challenges inherent in managing distributed evolution while retaining overall system coherence [16, 18].

6.3 Approaches for ensuring consistency

RQ1: What are the current architectural approaches to ensure consistency in event-driven microservices?

Following the identification of the three consistency types, approaches to addressing them can be organized along the same categories. The literature proposes a variety of architectural strategies, frameworks, and tools to mitigate these challenges, which are presented in the following sections.

6.3.1 Approaches to data consistency

Microservice architectures pose unique challenges for maintaining data consistency due to their independently managed services and heterogeneous data stores. Researchers have explored a range of approaches, from transactional models like ACID and Saga to asynchronous, event-driven, and backup-oriented frameworks. Each approach balances trade-offs between consistency, scalability, and reliability, reflecting the diverse strategies developed to manage data effectively in microservice environments.

1. *Final consistency model:* To tackle the challenge of maintaining data consistency in decentralized microservice architectures, this approach argues that traditional ACID transactions are unsuitable. Instead, it embraces the CAP theorem and adopts final consistency, ensuring data eventually aligns across services. Models such as reliable events, business compensation, and Try-Confirm-Cancel are used to coordinate updates asynchronously while maintaining availability and resilience. [27]. It is important to note that "eventual" consistency, as discussed in this approach, refers to the gradual alignment of distributed state over time and is unrelated to "events," despite the similarity in terminology.
2. *Proxy-based ACID transaction management:* In contrast to Fan, et al. [27], this approach claims that strong ACID guarantees can still be achieved in microservice systems through architectural innovation. The proposed Service Proxy Transaction Management method enables transparent, distributed ACID transactions by externally intercepting and managing service communications, preserving ACID consistency while offering scalable reads, without altering existing service code [29].
3. *Adopting Sagas:* To overcome the general limitations of ACID transactions, the approach proposed by [28] promotes the Saga pattern as a more practical solution for microservice architectures. Unlike the Two-Phase Commit protocol (2PC), Saga avoids data locking by breaking a transaction into smaller, local steps, each with its own compensating action.
4. *Semantic-based coordination service:* To address the limited guarantees of traditional Saga implementations, this approach introduces a coordination layer grounded in event-based, data-centric semantics. By modeling cross-service interactions with Interface Automata and validating event flows through runtime monitors, it ensures causal consistency and prevents anomalies that ad-hoc compensation logic may overlook [43].
5. *Asynchronous and buffered transaction models:* Some recent studies argue that neither strict ACID transactions nor the Saga pattern are suitable for modern, large-scale microservice systems. One approach, proposed for NoSQL-based big data environments, rejects ACID due to the inefficiency of the 2PC protocol and dismisses Saga for lacking isolation and causing anomalies under high loads [31]. It instead introduces an asynchronous transaction platform that coordinates operations across services without locking, maintaining data correctness even in failure-prone conditions. Similarly, another paper [9] identifies major shortcomings in both Saga, because of its dependence on compensating transactions that

may fail, and 2PC, which blocks resources and violates availability. To address these issues, it proposes a buffered serialization mechanism that caches transaction metadata to enable efficient recovery and retries after failure, offering a more reliable and performance-oriented consistency model.

6. *Rolling backup mechanism:* This paper examines the trade-off between backup, availability, and consistency in microservice systems. It proposes a clustered backup process where services perform staggered snapshots, allowing full-system recovery while preserving state integrity and minimizing downtime, thus maintaining consistency without sacrificing system availability [37].
7. *Event-driven broker architecture:* A hierarchical Edge–Cloud publish/subscribe model is introduced to improve communication efficiency and scalability in large-scale IoT microservice environments. By grouping brokers based on event channels and geolocation, the system reduces latency and relay traffic while maintaining reliable data exchange between distributed services [40].
8. *Containerized event-log architecture:* This approach improves synchronization and fault tolerance among microservices through an asynchronous, event-driven design. Each microservice runs in a container and reports to a central event manager that logs and coordinates inter-service communication, reducing response time and preventing data inconsistencies caused by missed or duplicate events [25].
9. *Database-centered coordination model:* Challenging the decentralized "database-per-service" principle, this approach introduces a centralized data governance layer, using database-level abstractions to represent service state and constraints. It shifts consistency and coordination responsibilities into the data platform, aiming to simplify cross-service consistency management [24].
10. *Scalable real-time stream processing:* To overcome latency and consistency issues in high-velocity data streams, this approach uses an event-driven pipeline with autoscaling triggered by Kafka lag rather than CPU usage, enabling the system to react to workload spikes without manual intervention [44]. A key mechanism for maintaining data consistency in this context is the standardization of timestamps across distributed data streams: raw Unix timestamps from ingested data are normalized to a common format, ensuring temporal alignment between sources and enabling correct ordering, comparison, and tracking of time-series data as it flows through the pipeline.

6.3.2 Approaches to process consistency

In event-driven microservice architectures, ensuring process consistency requires mechanisms to track and coordinate event flows. Key approaches include graph-based system comprehension and adaptations of orchestration and choreography models, which aim to improve visibility into control flows while maintaining service autonomy.

1. *Graph-based system comprehension:* To tackle the difficulty documenting information of the event-driven architectural style with existing diagrams (e.g. UML), visual graphs are used to expose system-wide behavior, recognizing communication

loops, and being able to analyze the event chains triggered from specific events [10]. Zhao, et al. [19] argue that traditional graphs group microservices mainly by what they do, but ignore the order in which services need to run, an important aspect for orchestration. To address this, they introduce IM-GNN, a graph neural network model that learns representations of microservices and compares them to suggest suitable orchestration candidates. Additionally, Directed Acyclic Graphs and Dynamic Condition Response Graphs are mentioned as mechanisms to support coordination, and the specification and analysis of requirements [41][3].

2. *BPMN-based coordination with dual composition views:* This approach is motivated by the difficulty of analyzing and evolving microservice choreographies, where control flow is distributed across autonomous services, hindering global reasoning and coordinated updates. To address this, it maintains two synchronized BPMN representations: i) a global model that provides a comprehensive view of the overall composition to support analysis and engineering decisions, and ii) a set of BPMN fragments executed by individual microservices via event-based choreography, preserving decoupling at runtime. This dual-view structure offers an additional benefit when composition requirements change, as updates can be performed locally on the affected fragments and then propagated to the global model through a synchronization algorithm, preserving consistency between both views during system evolution [3].
3. *Quality model for maintainability:* A hierarchical fuzzy logic model connects microservice sub-characteristics, such as modifiability and testability, to low-level static software metrics, enabling the prediction of maintainability during development. This model helps identify services with low maintainability that could threaten process-level consistency [50].
4. *Decentralized reactive architecture:* This model adapts the orchestration model to address time and mobility constraints in event streams for reactive microservices. Reactive microservices are composed of isolated components that respond to events asynchronously, either independently or in coordination with others, to achieve a system's intended outcomes. The proposed solution develops an event-driven coordination model that enables services to manage interactions autonomously through event streams, eliminating the need for centralized control and improving system resilience [41].
5. *Declarative orchestration with dynamic service discovery:* This approach addresses the challenge of dynamically locating microservices, as traditional orchestration typically requires prior registration of participating services. To overcome this limitation, it employs declarative business processes to support the modeling and execution of microservice orchestration from a data-flow perspective, enabling more flexible binding of services at runtime [22].

6.3.3 Approaches to system evolution consistency

Similar to the strategies for process consistency, approaches for system evolution consistency often leverage graph-based models and adaptations of orchestration and choreography in event-driven architectures. A distinguishing concept in this group is service discovery, which is central to identifying changes in the system, such as

the addition or removal of microservices, by determining the location of service instances at runtime [18]. Service registries play a key role in supporting discovery, as when new instances are deployed, their data models are registered to declare their availability [39].

1. *Graph-based system evolution:* Graph representations also appear as key contributors to guarantee a correct evolution of microservices architecture. One line of work models services and their interactions as directed graphs to enable lightweight declarative workflow planning, allowing automated workflow construction and flexible reconfiguration in evolving systems [16]. Another approach focuses on uncovering hidden semantic dependencies between microservices by constructing and comparing Component Call Graphs. This method identifies semantic similarities across services and visualizes them in a Semantic Dependency Matrix, supporting maintainability and impact analysis during system evolution [49].
2. *Event-driven orchestration platform:* A loosely coupled orchestration mechanism eliminates the need for static service registration by coordinating sequences of events and tasks across distributed microservices in a runtime-composable manner [18].
3. *Model-level change propagation:* This work proposes an evolution protocol to manage the propagation of changes from a local microservice perspective, preserving choreography integrity. The protocol relies on adaptation rules to automate suggested modifications and supports bottom-up evolution of BPMN process fragments [13]. The same authors later extend this approach by formalizing the previously informal adaptation rules and integrating them into a fully automated control mechanism. This extension broadens the scope from pure control-flow changes to also cover data-exchange evolution and introduces a dedicated monitoring service that detects local modifications and applies adaptations dynamically, a capability validated through expert comparison [45].
4. *Data-driven discovery:* Instead of accessing microservices by name, this approach allows client programs to discover relevant services based on their needs and data. It is particularly useful in dynamic environments where developers frequently publish new microservices that may otherwise go unnoticed, enabling clients to access available functionalities as they emerge [39].
5. *Eureka service registry:* The Eureka Server, an open-source service registry developed by Netflix, is frequently cited in the literature as a core mechanism for dynamic service discovery and registration [3]. It enables microservices to register themselves and exposes their endpoints through HTTP connections, allowing other services to locate and communicate with them efficiently. A key advantage of integrating Eureka into orchestration frameworks is that it eliminates the need for manual pre-registration of microservices, enabling new service instances to be discovered and incorporated dynamically during execution [18].

6.4 Business processes in microservice architectures

RQ2: How do these approaches incorporate business process concepts?

The findings for RQ1 connect closely with this question, as many of the consistency-preserving approaches were already found to implicitly or explicitly incorporate business process concepts. In fact, business process and event-driven paradigms frequently support the strategies used to ensure consistency in microservice systems across the reviewed studies. While not always the primary focus, these concepts often shape how services are modeled, coordinated, and evolved.

6.4.1 Design-time and runtime uses of process models

While business processes provide the conceptual foundation for modeling and coordinating service interactions, workflows represent their operational realization. Across the reviewed studies, this realization takes two qualitatively different forms depending on the role assigned to the business process model.

In one class of approaches, the business process model acts as a design-time source used to derive microservice structures, service contracts, or implementation artifacts. Jayawardana et al. [11] generate microservice code from a process specification that captures both process logic and the data entities each activity involves, making the process model the origin of the implementation. Once code generation is complete, the model and the deployed system are decoupled, with no mechanism maintaining alignment between them. Saidi et al. [51] use process structure earlier still, applying association rules mining to group activities that share data attributes into microservice candidates, with the consistency benefit arising from the decomposition decision itself.

In the other class, the business process model remains active at runtime as a coordination instrument. Monteiro et al. [22] execute declarative business processes through their proposed orchestration platform, which governs microservice coordination at runtime against explicit requirements for process compliance and data consistency. Importantly, both Monteiro et al. [22] and Valderas et al. [3] explicitly assume that microservices are independently developed and deployed beforehand. The process model is created to describe and govern their composition, not to derive them, which is a deliberate design choice: the business process is the coordination layer placed on top of existing services. Delgado et al. [47] present an intermediate case: the process model is used at design time to identify and derive microservices, but it is also subsequently executed in a business process management engine that invokes those microservices at runtime, keeping the model active across both phases.

The distinction between these roles matters for consistency: approaches that keep the process model active at runtime can monitor or enforce conformance as the system operates, while pure design-time approaches provide structural guarantees only at the point of generation and cannot detect subsequent drift.

6.4.2 Coordination and process integrity

Process modeling techniques such as BPMN are used to define and structure service interactions, and in several cases architectural boundaries are derived directly from business processes. Consistent with the discussion presented in previous sections,

orchestration, choreography, and hybrid coordination models emerge as the dominant paradigms in the reviewed studies.

At the design level, Noel et al. [48] go further than deriving service boundaries: their method analyses message structures in BPMN choreography diagrams to detect when a process participant manages a disproportionate share of domain knowledge, and provides guidelines to modularize participants and derive microservice domain models accordingly. The consistency argument is upstream: a poorly modeled process propagates domain coupling into microservice boundaries, increasing the risk of process consistency failures regardless of the coordination model subsequently chosen.

At the coordination level, Valderas et al. [3] and Ortiz et al. [13] both treat the global BPMN model as an active reference during execution. Valderas et al. maintain two synchronized representations, a global model and distributed BPMN fragments executed via event-based choreography, so the full process remains analyzable while services coordinate autonomously. Ortiz et al. extend this by addressing what happens when a local change in one fragment risks breaking the global composition: an adaptation protocol propagates the change and suggests compensating modifications to maintain process integrity. In both cases, the BPMN model is not left behind after design but remains the reference against which the running composition is assessed.

Finally, process observability through visual representations or dependency tracking supports understanding and managing complex distributed workflows, reinforcing the importance of transparency and traceability in maintaining consistency [10, 13].

6.4.3 The design-to-runtime gap

Looking across these works, a structural pattern emerges that constitutes the central finding for RQ2. Business process concepts appear in two distinct roles: as design-time sources that shape what gets built, and as runtime instruments that govern how existing services coordinate. The two groups are separated by a deliberate architectural boundary: runtime approaches explicitly assume services pre-exist and compose them via a process model, while design-time approaches derive services from a process model that subsequently plays no coordination role. No single paper connects both. The business process model never simultaneously governs structural decisions and runtime behavior, leaving a gap between what it shapes and what it actively enforces.

6.5 Challenges and limitations of business process use

RQ3: What limitations and challenges are reported regarding these approaches?

6.5.1 Modeling notation

A fundamental challenge in integrating business processes within microservice architectures lies in the limitations of traditional modeling notations such as Business Process Model and Notation (BPMN). Although BPMN is widely adopted to describe

business workflows and guide implementation, it was not originally designed with the decentralized and data-centric nature of microservices in mind. Consequently, while BPMN effectively captures control flows and high-level interactions, it lacks the expressive capability to specify detailed data dependencies, input and output parameters, or service-specific constraints that are essential in microservice systems [11].

This inadequacy creates a disconnect between business analysts and developers. Analysts can model the logical progression of a business process but cannot define the precise data requirements that govern each service interaction. Developers must therefore independently design message structures and data models that align with the intended business logic, introducing redundancy and the risk of semantic inconsistencies across services. Moreover, BPMN's strong alignment with orchestration-based control flows poses additional challenges in microservice contexts, where decentralized coordination and loose coupling are preferred. As noted by Valderas, et al. [3], current BPMN tools offer limited support for decomposing a process model into fragments that can be distributed across multiple microservices while maintaining synchronization between the overall process and its constituent parts.

6.5.2 Composition and coordination

When multiple microservices collaborate to execute a complex business process, explicit coordination mechanisms are required. This coordination is typically realized through orchestration or choreography, each introducing distinct challenges and trade-offs.

In orchestration, a central coordinator manages the sequence of service interactions. While this approach provides clearer control and process visibility, it also introduces a single point of failure and increases system complexity by adding an additional component that must tightly couple with all participating services [9, 29]. Furthermore, orchestrations often rely on synchronous communication to execute compositions, creating high dependencies between the orchestrator and the involved microservices and thereby reducing overall flexibility [3].

In contrast, choreography distributes control across participating services, allowing each to react to events and coordinate behavior autonomously. However, this decentralization shifts the coordination burden to individual services, which must now handle both their core business logic and the additional coordination responsibilities arising from the compositions they belong to [3].

7 Discussion

This review identified three recurring dimensions of consistency in event-driven microservice architectures, data consistency, process consistency, and system evolution consistency, and examined how contemporary designs address each. The following discussion synthesizes those findings into practical trade-offs and design guidance RQ1 and examines how business-process concepts are incorporated or under-utilized in current solutions RQ2. It focuses on three central decision points for architects:

transactional model selection (ACID vs Saga vs neither), coordination model selection (orchestration vs choreography vs hybrid), and the role of graph-based observability and tooling in managing implicit event flows.

7.1 ACID, Saga, or neither: trade-offs and when to choose each

The literature shows that ACID semantics remain desirable when systems must guarantee instantaneous, strongly consistent state across services, usually in narrowly scoped, tightly coupled domains where blocking and coordination costs can be tolerated (e.g., certain financial clearing scenarios). However, implementing distributed ACID across independently deployable microservices requires invasive coordination or architectural proxies that reintroduce coupling and operational complexity. Several authors argue these costs typically outweigh the benefits in large, elastic systems [27, 29].

By contrast, Saga patterns are widely advocated for long-running business processes that prioritize availability and decoupling, by breaking global transactions into local steps with compensating actions, enabling forward progress without global locks [28]. However, Saga introduces its own challenges: compensation logic is non-trivial, isolation is weaker, and subtle anomalies can arise unless coordination semantics are made explicit [9, 43]. Recent proposals therefore augment Sagas with semantic coordination layers and runtime monitors to validate causal dependencies and avoid anomalous states.

A third class of approaches rejects both ACID and Saga as unsuitable for large-scale, NoSQL-based microservice environments, proposing instead purpose-built asynchronous transaction platforms. González-Aparicio et al. [31] argue that 2PC is too inefficient for big data systems and that Saga lacks isolation guarantees under high loads. In response, they propose an asynchronous transaction platform that coordinates write and read operations across a cluster of NoSQL databases using a quorum-based protocol, with explicit fallback and rollback mechanisms to preserve consistency even in failure-prone conditions.

Practical decision criteria arise from these trade-offs, including the required invariants and their safety relevance, the system's tolerance for stale reads, acceptable failure modes, the need for auditability, and expected scale and latency. Therefore, Saga or event-log architectures are generally preferred in event-driven systems. Nevertheless, correctness still requires explicit compensation rules, runtime mechanisms that preserve causal order, and strategies that support recovery.

7.2 Choosing a coordination model: orchestration, choreography, or hybrid

Coordination decisions affect both process consistency and system evolvability. Orchestration centralizes control and therefore simplifies process visibility, enforcing a single source of sequencing and retry logic, making it relatively straightforward to implement system-wide transactional or compensating flows. However, the orchestrator becomes an architectural focal point whose failure modes, scalability and coupling must be carefully managed [9, 38].

Choreography preserves service autonomy and scales naturally with decentralized teams and partners, but its implicit flows can be hard to trace and reason about, which complicates debugging and evolution [10, 50]. BPMN choreography diagrams and similar artifacts can represent these interactions, but they typically lack sufficient data semantics to serve as executable contracts. Previous work also notes that choreography is suggested when only a small number of services participate in a transaction or when event chains remain relatively simple, whereas orchestration, despite its coordination overhead, becomes advantageous as transactional scenarios grow more complex. [23].

Hybrid approaches offer a pragmatic middle ground: orchestration for critical, tightly coordinated sequences and choreography for loosely coupled event propagation. The literature documents techniques that enable dynamic switching between centralized and distributed representations and use declarative business process fragments to orchestrate data flows without rigid pre-registration [3, 22]. These hybrid techniques reduce the trade-offs by harnessing orchestration where clarity and correctness are paramount while retaining choreography's resilience and loose coupling for routine event propagation.

When selecting a model architects should map the coordination choice to their consistency needs: orchestration for strong end-to-end control (at higher operational cost), choreography for scalability and autonomy (with investment in observability), and hybrid models when systems must balance the two.

7.3 Bridging process models and microservice implementations

Traditional modeling notations such as BPMN offer clear representations of control flow, but they struggle to capture the data semantics, interface contracts, and behavioral constraints that microservices depend on. As a result, analysts can describe high-level processes, yet developers still face ambiguity about payload structures, invariants, and service responsibilities, which contributes to drift between models and implementations [3, 11]. These limitations highlight the need for modeling approaches that better align with microservice principles by capturing both behavioral and data aspects of processes, while still supporting modularity and independent evolution [3, 13].

Recent work begins to address this gap by enriching models with semantic annotations, fragment-aware decomposition, and constraint specifications that guide verification and code generation. The continuing contributions across a series of works by Torres and Valderas with different collaborators [3, 13, 45] show how formal models and runtime artifacts can be gradually integrated to improve consistency management throughout the lifecycle. In this direction, Noel et al. [48] demonstrate how Domain-Driven Design principles can guide the derivation of microservice boundaries from process models, reducing the domain coupling that would otherwise propagate into runtime coordination problems. This connection between process modeling and domain-driven decomposition represents a promising path toward closing what the findings identified as the design-to-runtime gap: the absence of a process model that simultaneously governs structural decisions at design time and coordination behavior at runtime. Broader adoption of this direction will require tooling that keeps

models and implementations synchronized through automated contract derivation and feedback from runtime traces, ensuring they remain useful and trustworthy in evolving microservice architectures.

7.4 Graph-based observability and consistency monitoring

A recurring theme in the reviewed works is that implicit, event-driven interactions become opaque at scale. Here, graphs and advanced observability become central to recovering comprehension. Graph-based system models, ranging from directed call graphs to dynamic condition-response graphs and learned representations, make control and data flow explicit, enabling traceability, impact analysis for evolution, and automated recommendations for orchestration candidates [10, 19].

However, not all graph representations capture the aspects that matter in practice. Beyond showing which services interact, useful models need to express ordering, causality, and the meaning of the data that flows between components. These details become essential when checking invariants or understanding why a chain of events behaves the way it does. This richer level of description supports runtime monitoring, where causal consistency must be enforced, and it also supports system evolution, where hidden or fragile dependencies need to be identified.

Connecting this to the design-to-runtime gap identified in the findings, graph-based observability represents the technical layer most naturally positioned to bridge it. If the process model is represented as a graph and runtime event traces are similarly structured, it becomes possible to check whether an observed execution conforms to the intended business process, detecting deviations as they occur rather than after the fact. This is the direction that process mining and conformance checking have pursued in traditional process-aware systems, but it remains largely absent from the microservices literature reviewed here. Developing graph-based observability tools that are process-model-aware, rather than purely service-topology-aware, constitutes a concrete and well-grounded research direction that would directly address the fragmentation identified in this review.

7.5 Threats to validity

The possible threats to the validity are discussed in this section.

7.5.1 Internal validity

The main threat to internal validity in this review is the possibility of bias in the selection, extraction, and analysis of studies. To mitigate this, both authors were involved throughout the review process, jointly conducting screening, extraction, and analysis decisions, with disagreements resolved through discussion until consensus was reached. In addition, all candidate records were retrieved exclusively from the Web of Science database, which may have omitted relevant studies indexed in other digital libraries.

7.5.2 External validity

A potential threat to external validity is the relatively recent and evolving nature of microservices research. All included studies were published after 2017, not due to a filter but because the topic itself is still emerging. This means the findings of the review primarily reflect modern practices and perspectives on microservices and consistency, which may limit generalization to earlier software architectures or to contexts outside microservices. Additionally, the selected studies may not represent all industries or application domains equally (e.g. finance, healthcare, gaming), which could reduce the applicability of the findings across all software development environments.

7.5.3 Construct and conclusion validity

Construct validity could be threatened by differences in terminology across studies, as concepts such as "consistency" or "evolution" are not always used in the same way. To address this, we identified and separated these differences during analysis, grouping similar meanings before drawing conclusions. Conclusion validity is also affected by the varying quality and maturity of the selected studies, which range from empirical evaluations to more conceptual work. As a result, while the observed trends are meaningful, they should be interpreted with caution rather than as final or universal conclusions.

8 Conclusion

This paper reviewed the existing literature on consistency in microservice architectures, with particular attention to systems that integrate business process concepts. The motivation for the research arises from the rapid growth of event-driven microservices in industry, despite the absence of a globally accepted definition of microservices and the lack of a unified perspective on how consistency should be ensured in such architectures.

Through a literature review, three primary types of consistency concerns emerged as central in the current body of research: data consistency, which involves maintaining reliable and coherent data across distributed services, with common strategies including eventual consistency, event sourcing, and compensation mechanisms; process consistency, which addresses the alignment between business processes and service coordination, typically approached via workflow engines, choreography patterns, and orchestration techniques; and system evolution consistency, which refers to ensuring architectural coherence and stability during system changes, commonly supported through versioning strategies, service contract management, and governance models.

By synthesizing and organizing these three types of consistency into a unified perspective, this paper highlights a gap in current research: there is no comprehensive framework or tool for evaluating how well microservice-based systems address these consistency challenges. Existing works tend to focus on individual aspects in isola-

tion, making it difficult to systematically assess or compare architectures with respect to their overall consistency.

The findings underscore the importance of developing evaluation mechanisms that consider all three types of consistency, providing a structured basis for understanding, monitoring, and improving distributed systems. Such an approach could inform future research and practical efforts, enabling more robust, predictable, and coherent microservice architectures.

Author contributions All authors contributed equally to the manuscript.

Funding Open access funding provided by FCT|FCCN (b-on). No funding was received for conducting this study.

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Kitchenham, B (2004) Procedures for performing systematic reviews. Technical report, Keele University, Keele, Staffs. https://www.inf.ufsc.br/~aldo.vw/kitchenham.pdf
2. Tang C (2025) Meta's hyperscale infrastructure: Overview and insights. Commun ACM 68(2):52–63
3. Valderas P, Torres V, Pelechano V (2020) A microservice composition approach based on the choreography of BPMN fragments. Inf Softw Technol 127:106370. https://doi.org/10.1016/j.infsof.2020.106370
4. Liu G, Huang B, Liang Z, Qin M, Zhou H, Li Z (2020) Microservices: architecture, container, and challenges. In: 2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C), pp. 629–635. IEEE, Macau, China. https://doi.org/10.1109/QRS-C51114.2020.00107
5. Grønli T-M, Kazman R, Mikkelsen A, Tamburri DD (2020) Architectural principles for autonomous microservices. In: 53rd Hawaii International Conference on System Sciences, HICSS 2020, Maui, Hawaii, USA, January 7-10, 2020, pp. 1–10. ScholarSpace, Honolulu, HI, USA (2020). https://doi.org/10.24251/HICSS.2020.804
6. Taibi D, Lenarduzzi V, Pahl C (2017) Processes, motivations, and issues for migrating to microservices architectures: An empirical investigation. IEEE Cloud Comput 4(5):22–32. https://doi.org/10.1109/MCC.2017.4250931

7. Hernández-Aparicio CC, Ocharán-Hernández JO, Cortes-Verdin K, Arenas-Valdés MA (2022) Architectural languages for the microservices architecture: A systematic mapping study. In: 2022 10th International Conference in Software Engineering Research and Innovation (CONISOFT), pp. 192–201. IEEE, Ciudad Modelo, San José Chiapa, Mexico. https://doi.org/10.1109/CONISOFT55708.2022.00033
8. Gördesli M, Varol, A (2022) Comparing interservice communications of microservices for e-commerce industry. In: 2022 10th International Symposium on Digital Forensics and Security (ISDFS), pp. 1–4. IEEE, Istanbul, Turkey . https://doi.org/10.1109/ISDFS55398.2022.9800784
9. Munonye K, Martinek P (2020) Enhancing performance of distributed transactions in microservices via buffered serialization. J Web Eng https://doi.org/10.13052/jwe1540-9589.19564
10. Schoop S, Hebisch E, Franz T (2024) Improving comprehensibility of event-driven microservice architectures by graph-based visualizations. In: Galster, M., Scandurra, P., Mikkonen, T., Oliveira Antonino, P., Nakagawa, E.Y., Navarro, E. (eds.) Software Architecture vol. 14889, pp. 207–214. Springer, Cham. https://doi.org/10.1007/978-3-031-70797-1_14
11. Jayawardana Y, Fernando R, Jayawardena G, Weerasooriya D, Perera I (2018) A full stack microservices framework with business modelling. In: 2018 18th International Conference on Advances in ICT for Emerging Regions (ICTer), pp. 78–85. IEEE, Colombo, Sri Lanka. https://doi.org/10.1109/ICTER.2018.8615473
12. Abd-Elwahab AM, Mohamed AG, Shaaban EM (2023) Microservices-driven enterprise architecture model for infrastructure optimization. Future Bus J 9(1):90. https://doi.org/10.1186/s43093-023-00268-3
13. Ortiz J, Torres V, Valderas P (2023) Microservice compositions based on the choreography of BPMN fragments: facing evolution issues. Computing 105(2):375–416. https://doi.org/10.1007/s00607-022-01128-8
14. Alpers S, Becker C, Oberweis A, Schuster T (2015) Microservice based tool support for business process modelling. In: 2015 IEEE 19th International Enterprise Distributed Object Computing Workshop, pp. 71–78. https://doi.org/10.1109/EDOCW.2015.32
15. Geisriegler M, Kolodiy M, Stani S, Singer R (2017) Actor based business process modeling and execution: A reference implementation based on ontology models and microservices. In: 2017 43rd Euromicro Conference on Software Engineering and Advanced Applications (SEAA), pp. 359–362 . https://doi.org/10.1109/SEAA.2017.39
16. Oberhauser, R (2017) Microflows: Automated planning and enactment of dynamic workflows comprising semantically-annotated microservices. In: Shishkov, B. (ed.) Business Modeling and Software Design vol. 275, pp. 183–199. Springer, Cham . https://doi.org/10.1007/978-3-319-57222-2_9
17. Wiśniewski P, Kluza K, Jemioł o P, Ligeza A, Suchenia A (2021) Business process recomposition as a way to redesign workflows effectively. In: 2021 16th Conference on Computer Science and Intelligence Systems (FedCSIS), pp. 471–474. https://doi.org/10.15439/2021F138
18. Monteiro D, Gadelha R, Maia PHM, Rocha LS, Mendonça NC (2018) Beethoven: An event-driven lightweight platform for microservice orchestration. In: Cuesta, C.E., Garlan, D., Pérez, J. (eds.) Software Architecture vol. 11048, pp. 191–199. Springer, Cham. https://doi.org/10.1007/978-3-030-00761-4_13
19. Zhao T, Chen T, Sun Y, Xu Y (2025) Im-gnn: Microservice orchestration recommendation via interface-matched dependency graphs and graph neural networks. Symmetry **17**(4) https://doi.org/10.3390/sym17040525
20. Ta PB, Mafeni V, Kim Y (2025) A design of workflow-based automated failure recovery framework in iot edge environment. IEEE Access 13:101885–101899. https://doi.org/10.1109/ACCESS.2025.3578739
21. Trabelsi N, Politowski C, Boussaidi GE (2023) Event driven architecture: An exploratory study on the gap between academia and industry. In: 2023 IEEE/ACM 5th International Workshop on Software Engineering Research and Practices for the IoT (SERP4IoT), pp. 25–32 . https://doi.org/10.1109/SERP4IoT59158.2023.00010
22. Monteiro D, Maia PHM, Rocha LS, Mendonça NC (2020) Building orchestrated microservice systems using declarative business processes. Serv Oriented Comput Appl 14(4):243–268. https://doi.org/10.1007/s11761-020-00300-2
23. Rudrabhatla CK (2018) Comparison of event choreography and orchestration techniques in microservice architecture. Int J Adv Comput Sci Appl 9(8): https://doi.org/10.14569/IJACSA.2018.090804

24. Laigner R, Zhou Y, Salles MAV (2021) A distributed database system for event-based microservices. In: Proceedings of the 15th ACM International Conference on Distributed and Event-Based Systems, pp. 25–30. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3465480.3466919
25. Zuki SZM, Mohamad R, Saadon NA (2024) Containerized event-driven microservice architecture. Baghdad Sci. J. 21(2(SI)), 0584 https://doi.org/10.21123/bsj.2024.9729
26. Woodside, M (2021) Performance models of event-driven architectures. In: Companion of the ACM/SPEC International Conference on Performance Engineering, pp. 145–149. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3447545.3451203
27. Fan W, Han Z, Zhang Y, Wang R (2018) Method of maintaining data consistency in microservice architecture. In: 2018 IEEE 4th International Conference on Big Data Security on Cloud (BigDataSecurity), IEEE International Conference on High Performance and Smart Computing, (HPSC) and IEEE International Conference on Intelligent Data and Security (IDS), pp. 47–50. IEEE, Omaha, NE, USA. https://doi.org/10.1109/BDS/HPSC/IDS18.2018.00023
28. Lee W-T, Song P-Y, Tsai M-K (2025) A data-consistent microservices architecture library using saga design pattern and backup mechanism. J Inf Sci Eng 41(1):121–136. https://doi.org/10.6688/JISE.202501
29. Nikolic L, Dimitrieski V, Celikovic M (2024) An approach for supporting transparent ACID transactions over heterogeneous data stores in microservice architectures. Comput Sci Inf Syst 21(1):167–202. https://doi.org/10.2298/CSIS221210006N
30. Stefanko M, Chaloupka Or, Rossi B (2019) The saga pattern in a reactive microservices environment. In: Proceedings of the 14th International Conference on Software Technologies (ICSOFT 2019), pp. 483–490. https://doi.org/10.5220/0007918704830490
31. González-Aparicio MT, Younas M, Tuya J, Casado R (2023) A transaction platform for microservices-based big data systems. Simul Model Pract Theory 123:102709. https://doi.org/10.1016/j.simpat.2022.102709
32. Daraghmi E, Zhang CP, Yuan SM (2022) Enhancing saga pattern for distributed transactions within a microservices architecture. Appl. Sci. **12**(12) https://doi.org/10.3390/app12126242
33. Söylemez M, Tekinerdogan B, Koluk?sa Tarhan A (2022) Challenges and solution directions of microservice architectures: A systematic literature review. Applied Sciences **12**(11) https://doi.org/10.3390/app12115507
34. Laigner R, Zhou Y, Salles MAV, Liu Y, Kalinowski M (2021) Data Management in Microservices: State of the Practice, Challenges, and Research Directions. https://arxiv.org/abs/2103.00170
35. Laigner R, Almeida AC, Assunção WKG, Zhou Y (2025) An empirical study on challenges of event management in microservice architectures. ACM Trans Softw Eng Methodol. https://doi.org/10.1145/3776581. (**Just Accepted**)
36. Alshuqayran N, Ali N, Evans R (2016) A systematic mapping study in microservice architecture. In: 2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA), pp. 44–51. https://doi.org/10.1109/SOCA.2016.15
37. Pardon G, Pautasso C, Zimmermann O (2018) Consistent disaster recovery for microservices: the BAC theorem. IEEE Cloud Comput 5(1):49–59. https://doi.org/10.1109/MCC.2018.011791714
38. Rossi, D (2019) Consistency and availability in microservice architectures. In: Escalona, M.J., et al. (eds.) WEBIST 2018 vol. 372, pp. 39–55. Springer, Cham, Switzerland . https://doi.org/10.1007/978-3-030-35330-8_3
39. Houmani Z, Balouek-Thomert D, Caron E, Parashar M (2020) Enhancing microservices architectures using data-driven service discovery and qos guarantees. In: 2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID), pp. 290–299. https://doi.org/10.1109/CCGrid49817.2020.00-64
40. Pham V-N, Hossain MD, Lee G-W, Huh E-N (2023) Efficient data delivery scheme for large-scale microservices in distributed cloud environment. Appl Sci 13(2):886. https://doi.org/10.3390/app13020886
41. Goel D, Nayak A (2019) Reactive microservices in commodity resources. In: 2019 IEEE International Conference on Big Data (Big Data), pp. 3658–3665. IEEE, Los Angeles, CA, USA. https://doi.org/10.1109/BigData47090.2019.9006584
42. Ortiz G, Caravaca JA, Garcia-de-Prado A, Chavez De La O F, Boubeta-Puig J (2019) Real-time context-aware microservice architecture for predictive analytics and smart decision-making. IEEE Access 7, 183177–183194 https://doi.org/10.1109/ACCESS.2019.2960516

43. Zuckmantel T, Zhou Y, Düdder B, Hildebrandt T (2022) Event-based data-centric semantics for consistent data management in microservices. In: Proceedings of the 16th ACM International Conference on Distributed and Event-Based Systems, pp. 97–102. Association for Computing Machinery, New York, NY, USA . https://doi.org/10.1145/3524860.3539807
44. Reddy DN, Suryodai R, B VKS, Ambika M, Muniyandy E, Krishna VR, Abdurasul B (2025) A scalable microservices architecture for real-time data processing in cloud-based applications. International Journal of Advanced Computer Science and Applications **16**(9) https://doi.org/10.14569/IJACSA.2025.0160905
45. Ortiz J, Torres V, Valderas P (2025) Supporting the evolution of event-based choreographies of bpmn fragments in microservices environments. IEEE Access 13:137827–137849. https://doi.org/10.1109/ACCESS.2025.3595387
46. Li W, Li Z, Yan Z, Liu Y, Zeng D, Yu H, Chen W, Wu F (2025) A data encryption and file sharing framework among microservices-based edge nodes with blockchain. Peer-to-Peer Netw Appl 18(1):24. https://doi.org/10.1007/s12083-024-01856-y
47. Delgado A, García F, Ruiz F (2023) Bizdevops support for business process microservices-based applications. In: Troya, J., Mirandola, R., Navarro, E., Delgado, A., Segura, S., Ortiz, G., Pautasso, C., Zirpins, C., Fernández, P., Ruiz-Cortés, A. (eds.) Service-Oriented Computing – ICSOC 2022 Workshops vol. 13821, pp. 274–286. Springer, Cham. https://doi.org/10.1007/978-3-031-26507-5_22
48. Noel R, España S, Panach JI, Pastor O (2024) Microservices-aware business process modelling. In: Marrella, A., Resinas, M., Jans, M., Rosemann, M. (eds.) Business Process Management Forum vol. 526, pp. 267–284. Springer, Cham. https://doi.org/10.1007/978-3-031-70418-5_16
49. Abdelfattah AS, Cordes KE, Medina A, Cerny T (2025) Semantic Dependency in Microservice Architecture: A Framework for Definition and Detection. arXiv:2501.11787 . https://arxiv.org/abs/2501.11787
50. Yilmaz R, Buzluca F (2024) A fuzzy logic-based quality model for identifying microservices with low maintainability. J Syst Softw 216:112143. https://doi.org/10.1016/j.jss.2024.112143
51. Saidi M, Daoud M, Tissaoui A, Sabri A, Benslimane D., Faiz S (2022) Automatic microservices identification from association rules of business process. In: Abraham, A., Gandhi, N., Hanne, T., Hong, T.-P., Nogueira Rios, T., Ding, W. (eds.) Intelligent Systems Design and Applications vol. 418, pp. 476–487. Springer, Cham . https://doi.org/10.1007/978-3-030-96308-8_44
52. Tapia VC, Gaona CM (2023) Research opportunities in microservices quality assessment: A systematic literature review. J Adv Inf Technol 14(5):991–1002. https://doi.org/10.12720/jait.14.5.991-1002
53. Sheyanov NN, Sinev MP, Neshko DO, Pashchenko DV, Trokoz DA, Martyshkin AI (2024) Methods to synchronize data in a microservice architecture. Sci Iran 31(22):2088–2095. https://doi.org/10.24200/sci.2022.53164.3088

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Joaquim Luz Bação[1] · Sérgio Guerreiro[1,2]

✉ Joaquim Luz Bação
joaquimluzbacao@tecnico.ulisboa.pt

Sérgio Guerreiro
sergio.guerreiro@tecnico.ulisboa.pt

[1] Department of Computer Science and Engineering, Instituto Superior Técnico, Universidade de Lisboa, Lisboa, Portugal

[2] Information and Decision Support Systems (IDSS), Instituto de Engenharia de Sistemas e Computadores - Investigação e Desenvolvimento em Lisboa (INESC-ID), Lisboa, Portugal